

# Practical Task 8.1

(Credit Task)

Submission deadline: Monday, May 25

Discussion deadline: Friday, June 5

## General Instructions

This practical task asks you to extend the functionality of the generic [data structure MyStack<T>](#) considered as a demo program in [Lecture 10](#).

1. To start your work, create a new project and import the C# code of MyStack<T> class attached to this task. Rename the default Program class to Tester; you will use it to debug and test the extended version of your [stack data structure](#).
2. You now need to add and accurately implement the following **generic public** methods as an extension of the MyStack<T> class.

- **T Find( Func<T, bool> criteria )**

Searches for the first occurrence of an element in the MyStack<T> data structure that satisfies the condition specified by the delegate *criteria* fed as the input parameter. The search starts at the top of the respective stack. If there is no element meeting the search criteria, the method returns the default value for type T, which is the [Default\(T\)](#). This method throws the [ArgumentNullException](#) if the given *criteria* appears null.

- **T[] FindAll( Func<T, bool> criteria )**

Retrieves all elements that satisfy the condition specified by the delegate *criteria* fed as the input parameter. If there are elements that meet the condition, this method returns them stored in a type T array; otherwise, this method returns null to signal that no elements exist. This method throws the [ArgumentNullException](#) if the given *criteria* appears null.

- **int RemoveAll( Func<T, bool> criteria )**

Removes all elements stored in the MyStack<T> data structure that satisfy the condition specified by the delegate *criteria* fed as the input parameter. This method returns the number of elements removed from the MyStack<T>. If the given *criteria* appears null, this method throws the [ArgumentNullException](#).

- **T Max( )**

Returns an element with the maximum value stored in the respective MyStack<T>. If the MyStack<T> is empty, it returns the default value for type T, which is the [Default\(T\)](#).

- **T Min( )**

Returns an element with the minimum value stored in the respective MyStack<T>. If the MyStack<T> is empty, it returns the default value for type T, which is the [Default\(T\)](#).

You may need to do extra research about the purpose of [Default\(T\)](#) to get more details about what is known as a default value for generic type T. Furthermore, to understand how `Func<T, bool> criteria` variable should act in the Find, FindAll, and RemoveAll methods, you should go through the lecture material of weeks 8 and 9 and do additional research about anonymous [delegates and Lambda expressions](#).

In order to implement the Min and Max methods properly, use the Comparer<T> type object (e.g., **comparer**) that you may obtain from the [Default](#) property of the Comparer<T> class as follows:

```
Comparer<T> comparer = Comparer<T>.Default;
```

In this example, the **comparer** object will allow you to call its [Compare\(T, T\)](#) method in order to compare two variables of type T to determine whether one variable is greater than, equal to, or smaller than the other. To get more information about the [Default](#) property, check the description of [Comparer<T>](#) class.

3. As you progress with the implementation of the methods listed above, you need to test each of them. For this purpose, use the `Account` class that you have completed in Task 3.2. Import the class to your current project. You need to slightly extend it by adding one more property required to get the current account balance; that is, add the following:

- **decimal Balance**

Property. Gets the `_balance` of its account.

4. Test the `MyStack<T>` class and ensure that you cover all potential logical issues and runtime errors. This (testing) part of the task is as important as writing code for this data structure. Specifically, use your `Tester` class to create and populate a stack of accounts (no input from the user is required; make your example hard coded). Then, you may check against multiple test cases, for example:
  - Check whether you can find an account with a specific balance. Test the `Find` method for both variants: one where such account does exist in the stack, and the other where there is no appropriate account. You should examine different configurations of the search criteria: Search for the first account with a strictly positive balance; or with a strictly positive balance that does not exceed \$100.
  - Check whether you can, using the `FindAll` method, find all accounts with a specific balance and name. Also, does this method properly find all accounts that are, for example, related to credit cards and short term deposits (use the `Name` property to determine the account type)?
  - Similarly to the test of the `FindAll` method, check whether the `RemoveAll` method meets the given description and works correctly in regard to the input criteria.

Make sure you test how these three methods work when the respective input criteria is null; that is, do they throw the expected exception?

5. To test the `Min` and the `Max` methods, you first must make objects of the `Account` class comparable. Specifically, you need to implement a mechanism that compares two elements of the `Account` type, and thus provides a default way to determine the relation between these elements. Fortunately, .NET Framework has a standard way to realize this approach: The `Account` class must implement the special [IComparable<T> interface](#). The `IComparable<T>` interface imposes implementation of the compulsory [CompareTo\(T other\)](#) method that compares [this \(current\) element](#) to the other element (given as the input argument) and returns an integer value that is
  - less than zero, when [this instance \(object\)](#) is considered smaller than the other object specified in the input of the `CompareTo` method;
  - zero, when this instance (object) is considered equal to the other; and
  - greater than zero, when this instance (object) is considered greater than the other object.

To get details on how to apply the [IComparable<T> interface](#), you will need to do some extra research.

It is important to note that in regard to the `Account` class, the generic type `T` is a placeholder that is to be replaced by the `Account` as an actual class. Specifically, you will need to modify the `Account` class and implement the `IComparable<Account>` interface along with its underlying `CompareTo(Account other)` method. Then, the `CompareTo(Account other)` must return

- a negative integer (e.g., -1), when the balance of this `Account` object is smaller than the balance of the other account;
  - zero, when the balance of this `Account` type object is the same as the balance of the other account; and
  - a positive integer (e.g., 1), when the balance of this `Account` type object is larger than the balance of the other account.
6. Now, when the `Account` class is ready, you may proceed with testing of the `Min` and the `Max` methods of the `MyStack<T>` class. Use your main `Tester` class to check that these methods return accounts with the minimum and the maximum balance, respectively.

## Further Notes

- This practical task is based on [the material of lectures 7-10](#). You should watch them to complete the task.
- Explore Section 11.3 of the SIT232 Workbook to learn more about Generics and their application. The workbook is available in CloudDeakin → Learning Resources.
- The following tutorial will give you more details about delegates and Lambda expressions
  - <https://www.codeproject.com/Articles/47887/C-Delegates-Anonymous-Methods-and-Lambda-Expressio>
- To get more insights on generics and their application, you may wish to check the following links:
  - <https://www.c-sharpcorner.com/article/defaultt-in-generics/>
  - <https://www.geeksforgeeks.org/c-sharp-generics-introduction/>
  - <https://dotnettutorials.net/lesson/generics-csharp/>
- Explore the following links to learn more about the Comparer<T> class and the IComparable<T> interface used in this task:
  - <https://docs.microsoft.com/en-us/dotnet/api/system.collections.generic.comparer-1>
  - <https://docs.microsoft.com/en-us/dotnet/api/system.collections.generic.comparer-1.default>
  - <https://docs.microsoft.com/en-us/dotnet/api/system.collections.generic.comparer-1.compare>
  - [https://msdn.microsoft.com/en-us/library/system.icomparable\(v-vs.110\).aspx](https://msdn.microsoft.com/en-us/library/system.icomparable(v-vs.110).aspx)
  - <https://docs.microsoft.com/en-us/dotnet/api/system.icomparable-1.compareto>

## Submission Instructions and Marking Process

To get your task completed, you must finish the following steps strictly on time.

- Make sure your programs implement the required functionality. They must compile and have no runtime errors. Programs causing compilation or runtime errors will not be accepted as a solution. You need to test your programs thoroughly before submission. Think about potential errors where your programs might fail.
- **Submit** the expected code files as a solution to the task via OnTrack submission system. You must **record a short video explaining your solution** to the task. Upload the video to one of accessible resources and refer to it for the purpose of marking. You must provide a private working link to the video to your marking tutor in OnTrack. Note that the video recording must be made in the **camera on mode**; that is, the tutor must see both the presenter and the shared screen.
- Once your solution is accepted by the tutor, you will be invited to **continue its discussion and answer relevant theoretical questions** through the Intelligent Discussion Service in OnTrack. Your tutor will record several audio questions. When you click on these, OnTrack will record your response live. You must answer straight away in your own words. As this is a live response, you should ensure you understand the solution to the task you submitted. Answer all additional questions that your tutor may ask you. Questions will cover the lecture notes; so, attending (or watching) the lectures should help you with this **compulsory discussion part**. You should start the discussion as soon as possible as if your answers are wrong, you may have to pass another round, still before the deadline. Use available attempts properly.

Note that we will not accept your solution after **the submission deadline** and will not discuss it after **the discussion deadline**. If you fail one of the deadlines, you fail the task, and this reduces the chance to pass the unit. Unless extended for all students, the deadlines are strict to guarantee smooth and on-time work throughout the unit.

Remember that this is your responsibility to keep track of your progress in the unit that includes checking which tasks have been marked as completed in the OnTrack system by your marking tutor, and which are still to be finalised. When grading your achievements at the end of the unit, we will solely rely on the records of the OnTrack system and feedback provided by your tutor about your overall progress and the quality of your solutions.